

SEGURIDAD Y RESILIENCIA — DARK IMPORTS

Documentación de seguridad, manejo de errores y Human-in-the-Loop
Make + Airtable + OpenAI + Telegram

1. Objetivo de seguridad y resiliencia

El objetivo es que el ecosistema pueda procesar consultas de clientes de forma automática sin comprometer credenciales, enviar información innecesaria a servicios externos ni continuar una operación cuando existan datos inválidos o una falla de API.

La arquitectura aplica tres principios: minimización de datos, fallos controlados y validación humana antes de acciones críticas.

2. Minimización de datos

OpenAI recibe únicamente la información necesaria para interpretar la consulta y generar la respuesta. Los datos estructurados del inventario permanecen en Airtable y no se envían campos irrelevantes.

Dato	¿Se utiliza?	Tratamiento
Consulta del cliente	Sí	Se envía a IA para interpretar intención y producto.
ID de cliente / chat	Sí	Se conserva para responder al usuario correcto.
Producto	Sí	Se utiliza para identificar el registro de inventario.
Precio / stock	Sí	Se envía como contexto comercial necesario.
Credenciales / API Keys	No	Nunca se incorporan al prompt ni a los mensajes.
Datos personales innecesarios	No	No se envían a OpenAI si no son necesarios para resolver la consulta.

3. Protección de credenciales

Las API Keys, tokens y credenciales deben mantenerse dentro del sistema de credenciales de Make y nunca escribirse directamente en módulos, prompts, Airtable, capturas de pantalla o repositorios públicos.

Para la entrega en GitHub, se deben excluir credenciales reales y utilizar valores ficticios o variables como YOUR_API_KEY. Las capturas deben ocultar tokens, IDs sensibles y cualquier secreto.

Riesgo	Medida
API Key expuesta	Usar Credentials de Make y ocultar secretos en screenshots.
Credencial en GitHub	No incluir tokens en JSON/Blueprint; reemplazarlos por placeholders.
Datos excesivos enviados a IA	Enviar solo variables necesarias para resolver la consulta.
Acceso indebido a Airtable	Compartir únicamente una vista de lectura cuando corresponda.

4. Gestión de errores y resiliencia

El flujo debe evitar que un error de un servicio externo termine en una respuesta incorrecta al cliente. Cada punto crítico debe tener una ruta de recuperación o registro del error.

Punto de fallo	Respuesta del sistema	Resultado
Telegram no entrega datos	Validar campos obligatorios antes de continuar.	No se procesa la consulta.
Airtable no encuentra producto	Router deriva a ruta PRODUCTO NO DISPONIBLE / CONSULTA AMBIGUA.	No se inventa stock ni precio.
OpenAI falla / timeout	Error Handler registra el fallo y evita enviar una respuesta generada incompleta.	Flujo estable.
Airtable falla al actualizar	Registrar error y detener la confirmación final.	No se declara procesado algo que no fue guardado.
Telegram falla al enviar	Registrar error y permitir reintento controlado.	Evita pérdida silenciosa del mensaje.

5. Error Handler

En Make, los módulos críticos deben utilizar una ruta de manejo de errores con directivas Resume o Break según el caso.

Resume: continuar cuando el error pueda recuperarse sin comprometer la integridad del proceso.

Break: detener la ejecución cuando continuar podría provocar una respuesta incorrecta, duplicada o una pérdida de datos.

Cuando un error no puede recuperarse, se recomienda registrar como mínimo: ID de ejecución, módulo que falló, tipo de error, mensaje, fecha/hora y estado de resolución.

6. Human-in-the-Loop (HITL)

El sistema debe detenerse antes de una acción crítica que pueda generar un compromiso comercial o una respuesta sensible. En ese punto se envía una solicitud de aprobación al responsable y la automatización espera su decisión.

Etapas	Acción	Control humano
1. Detectar caso crítico	La IA identifica negociación, reclamo, consulta ambigua o situación fuera del catálogo.	Se marca requiere_humano = true.
2. Pausar flujo	Make no continúa con la acción final.	La ejecución queda pendiente de aprobación.
3. Solicitar aprobación	Se envía una notificación al responsable.	Aprobar / rechazar / modificar.
4. Ejecutar	Solo después de aprobación se realiza la acción crítica.	Se registra la decisión.

7. Filtros anti-bucle y duplicados

El flujo debe contener controles que impidan ejecuciones repetidas o bucles infinitos.

- Usar un identificador único de mensaje o ejecución.
- Consultar el estado antes de procesar nuevamente una consulta.
- No disparar el mismo trigger como consecuencia de una actualización que el propio flujo generó sin un filtro.
- Utilizar rutas mutuamente excluyentes en el Router.
- Evitar que una respuesta enviada genere nuevamente una consulta procesable.

8. Validación de tipos de datos

Los filtros deben comparar valores del mismo tipo. Por ejemplo, si STOCK está almacenado como texto u opción "SI/NO", el filtro debe comparar contra el mismo tipo y no contra un booleano true/false sin conversión.

Campo	Tipo	Comparación correcta
STOCK	Texto / opción	STOCK = "SI"
APROBADO	Booleano	APROBADO = true
PRECIO	Número	PRECIO > 0
ID CLIENTE	Texto / número según configuración	Comparar manteniendo el mismo tipo

9. Recuperación ante fallos

La estrategia de recuperación prioriza la integridad de los datos sobre la continuidad a cualquier costo. Si el sistema no puede confirmar que dispone de información correcta, no debe inventar una respuesta ni marcar la consulta como procesada.

Flujo de recuperación recomendado:
Fallo → detectar → registrar → intentar recuperación cuando sea seguro → si persiste, detener → derivar a revisión humana.

10. Evidencias que conviene mostrar en la entrega

Para demostrar el criterio de seguridad y resiliencia, las capturas deben evidenciar:

- Ruta de Error Handler en Make.
- Filtro que evita procesar datos incompletos.
- Router con rutas de éxito y fallo.
- Punto donde el flujo espera aprobación humana.
- Credenciales ocultas.
- Registro de estados en Airtable.
- Ejecución de una prueba con datos incompletos o una ruta de error.

11. Resumen de arquitectura resiliente

Entrada: Telegram recibe la consulta → Validación: Make comprueba datos → Memoria: Airtable registra y consulta inventario → IA: OpenAI interpreta con datos mínimos → Router: separa rutas según disponibilidad y criticidad → HITL: detiene casos críticos para aprobación → Salida: Telegram responde → Error Handler: registra y recupera fallos sin enviar respuestas incorrectas.